Agentic RAG 정리

1. Agentic RAG란?

Agentic RAG는 기존 RAG에 **Agent의 판단과 실행 능력**을 결합한 방식이다.

일반 RAG는 보통 정해진 파이프라인을 따른다.

```
질문 → 검색 → 관련 문서 chunk → LLM 답변
```

Agentic RAG는 이 흐름을 고정하지 않는다.

```
질문 → 에이전트가 계획 수립 → 필요한 도구 선택 → 검색/검증/추론 반복 → 답변 생성
```

즉, 일반 RAG가 "한 번 검색해서 답하는 시스템"이라면, Agentic RAG는 "에이전트가 스스로 어떤 검색을 할지, 어떤 도구를 쓸지, 추가 검색이 필요한지 판단하며 답하는 시스템"이다.

쉽게 말하면:

> 일반 RAG는 검색 엔진에 가깝고,
> Agentic RAG는 조사원이나 리서치 어시스턴트에 가깝다.

---

2. 왜 필요한가?

기존 RAG는 단순한 질문에는 잘 작동한다.

예:

> "이 문서의 환불 정책은 뭐야?"

하지만 다음과 같은 질문은 더 복잡하다.

> "이 고객사의 최근 이슈를 정리하고, 원인과 해결 방안을 제안해줘."
> "이 논문들과 관련 문서를 비교해서 어떤 방법론이 더 나은지 판단해줘."
> "문서 A의 주장과 문서 B의 수치가 모순되는지 확인해줘."
> "내부 정책, 계약서, 티켓을 모두 참고해서 이 요청을 승인해도 되는지 검토해줘."

이런 질문은 단순 검색 한 번으로 끝나지 않는다.

필요한 작업은 보통 다음과 같다.

- 질문을 하위 문제로 나누기
- 어떤 문서를 찾아야 하는지 판단하기
- 여러 검색 쿼리를 생성하기

- 검색 결과가 충분한지 평가하기
- 부족하면 다시 검색하기
- 서로 다른 출처를 비교하기
- 모순을 확인하기
- 최종 답변을 구조화하기

Agentic RAG는 이 과정을 에이전트가 수행하도록 만든다.

---

3. 핵심 개념

Agentic RAG의 핵심은 **LLM이 단순 생성기가 아니라 오케스트레이터 역할을 한다는 것**이다.

일반 RAG에서 LLM은 주로 마지막 단계에서 답변을 생성한다.

```
Retriever → Context → LLM Answer
```

Agentic RAG에서 LLM은 중간 과정에도 개입한다.

```
LLM Agent
  ├── 질문 분석
  ├── 검색 필요 여부 판단
  ├── 검색 도구 선택
  ├── 쿼리 생성
  ├── 결과 평가
  ├── 추가 검색 여부 판단
  ├── 다른 도구 호출
  └── 최종 답변 생성
```

따라서 Agentic RAG는 단순한 검색 파이프라인이 아니라 **검색·판단·도구 사용·반복 추론을 포함한 워크플로우**다.

---

4. 일반 RAG와 Agentic RAG 비교

구분	일반 RAG	Agentic RAG
흐름	고정된 파이프라인	동적으로 결정되는 워크플로우
검색 횟수	보통 1회	필요하면 여러 번
쿼리 생성	사용자 질문 그대로 또는 단순 변환	에이전트가 하위 질문과 검색 쿼리 생성
도구 사용	주로 retriever	retriever, DB, API, 계산기, 웹, 코드 실행 등
강점	빠르고 단순함	복잡한 조사, 비교, 검증에 강함
약점	복잡한 질문에 취약	비용, 지연, 통제 난이도 증가
적합한 질문	“정책이 뭐야?”	“정책, 계약, 이슈를 종합해 판단해줘.”

5. Agentic RAG의 기본 동작 흐름

5.1 질문 분석

에이전트가 사용자 질문을 분석한다.

예:

> “ACME 고객사의 온보딩 지연 원인과 해결 방안을 정리해줘.”

에이전트는 이 질문을 다음처럼 나눌 수 있다.

```
1. ACME 고객사는 누구인가?
2. ACME와 관련된 최근 이슈는 무엇인가?
3. 온보딩 지연의 직접 원인은 무엇인가?
4. 관련 기능, 담당 팀, blocker는 무엇인가?
5. 해결 방안은 무엇인가?
```

5.2 계획 수립

에이전트가 어떤 순서로 조사할지 정한다.

```
Plan:
1. 고객 관련 문서 검색
2. 최근 티켓 검색
3. 회의록에서 blocker 검색
4. 제품 로드맵에서 관련 기능 상태 확인
5. 원인과 해결책 종합
```

5.3 도구 선택

에이전트는 질문에 따라 적절한 도구를 선택한다.

예:

```
문서 검색 → Vector Retriever
정확한 값 확인 → SQL DB
최근 이슈 확인 → 티켓 시스템 API
계산 필요 → Calculator
외부 정보 확인 → Web Search
코드 검증 → Code Interpreter
```

5.4 검색과 관찰

에이전트가 도구를 호출하고 결과를 관찰한다.

```
Tool call: search_docs("ACME onboarding delay SSO")
Observation: ACME onboarding is blocked by SSO configuration and security
review.
```

5.5 결과 평가

에이전트는 검색 결과가 충분한지 판단한다.

```
현재 정보:
- SSO 관련 blocker 확인됨
- Security Review 관련 정보 있음
- 담당 팀 정보 부족

다음 행동:
- Platform Team 관련 회의록 추가 검색
```

5.6 반복 검색

필요하면 추가 검색을 한다.

```
search_docs("Platform Team SSO priority ACME")
search_tickets("ACME SSO onboarding blocker")
search_roadmap("SSO enterprise roadmap")
```

5.7 최종 답변 생성

충분한 정보를 모으면 답변을 생성한다.

```
원인:
1. SSO 설정 미완료
2. Security Review 지연
3. Platform Team 우선순위 충돌

해결 방안:
1. ACME 전용 임시 설정 가이드 제공
2. Security Review SLA 지정
3. Platform Team sprint 우선순위 조정
```

---

6. Agentic RAG의 구성 요소

6.1 Agent

전체 흐름을 제어하는 핵심 주체다.

Agent는 다음을 수행한다.

- 질문 이해
- 계획 수립
- 도구 선택
- 중간 결과 평가
- 추가 검색 여부 판단
- 최종 답변 생성

6.2 Tools

Agent가 사용할 수 있는 외부 기능이다.

예:

```
retrieve_documents(query)
search_web(query)
query_sql(sql)
get_ticket(ticket_id)
run_python(code)
call_internal_api(endpoint)
```

Agentic RAG에서는 retriever도 여러 도구 중 하나가 된다.

6.3 Memory 또는 State

Agent가 작업 중간 상태를 저장하는 공간이다.

예:

```
현재 질문
검색한 문서 목록
찾은 근거
아직 모르는 정보
다음 행동 계획
중간 결론
```

LangGraph 같은 프레임워크에서는 이런 상태를 그래프의 state로 관리하는 경우가 많다.

6.4 Retriever

문서 검색을 담당한다.

Agentic RAG에서는 retriever가 하나만 있는 것이 아니라 여러 개일 수 있다.

예:

```
policy_retriever
contract_retriever
ticket_retriever
meeting_notes_retriever
codebase_retriever
```

Agent는 질문에 맞는 retriever를 선택한다.

6.5 Evaluator 또는 Critic

검색 결과나 답변 품질을 평가하는 역할이다.

예:

```
검색 결과가 질문에 충분한가?
출처가 신뢰할 만한가?
서로 모순되는 정보가 있는가?
답변에 근거가 포함되어 있는가?
추가 검색이 필요한가?
```

Evaluator는 별도 LLM 호출로 만들 수도 있고, Agent 내부 판단으로 처리할 수도 있다.

---

7. Agentic RAG의 대표 패턴

7.1 Query Planning

사용자의 질문을 여러 하위 질문으로 나누는 방식이다.

질문:

> “Graph RAG와 Agentic RAG의 차이를 설명하고, 우리 사내 문서 검색에는 어떤 게 나은지 판단해줘.”

하위 질문:

```
1. Graph RAG란 무엇인가?
2. Agentic RAG란 무엇인가?
```

```
3. 둘의 차이는 무엇인가?
4. 사내 문서 검색 요구사항은 무엇인가?
5. 어떤 방식이 더 적합한가?
```

복잡한 리서치 질문에 유용하다.

7.2 Tool Routing

질문에 맞는 도구를 선택하는 방식이다.

예:

```
정책 질문 → policy_retriever
계약 질문 → contract_retriever
고객 이슈 질문 → ticket_retriever
수치 분석 질문 → SQL 또는 Python
최신 외부 정보 → web_search
```

일반 RAG에서는 하나의 retriever만 쓰는 경우가 많지만, Agentic RAG에서는 여러 도구 중 적절한 것을 고른다.

7.3 Iterative Retrieval

검색 결과가 부족하면 다시 검색하는 방식이다.

```
검색 1: ACME onboarding issue
결과: SSO 문제 확인

검색 2: ACME SSO security review
결과: 보안 검토 지연 확인

검색 3: Platform Team SSO roadmap
결과: 담당 팀과 일정 확인
```

복잡한 문제는 한 번의 검색으로 충분한 근거가 나오지 않기 때문에 반복 검색이 중요하다.

7.4 Self-Reflection

에이전트가 자신의 중간 답변을 검토하는 방식이다.

예:

```
현재 답변에 원인은 포함되어 있지만, 해결 방안의 근거가 부족하다.
추가로 roadmap 문서를 검색해야 한다.
```

7.5 Corrective RAG

검색 결과가 부정확하거나 부족하면 쿼리를 수정하거나 다른 검색 경로를 선택하는 방식이다.

```
초기 검색 결과가 질문과 관련 없음
→ 쿼리 재작성
→ 다른 retriever 사용
→ 검색 범위 확장
```

7.6 Multi-Agent RAG

여러 에이전트가 역할을 나누어 협업하는 방식이다.

예:

```
Research Agent: 자료 수집
Analysis Agent: 비교와 분석
Critic Agent: 오류와 근거 검토
Writer Agent: 최종 답변 작성
```

대규모 리서치나 보고서 생성에 사용할 수 있다.

---

8. Agentic RAG 예시

8.1 일반 RAG의 답변

질문:

> “ACME 고객사의 온보딩 지연 원인은 뭐야?”

일반 RAG 흐름:

```
질문 → 벡터 검색 → 관련 chunk 5개 → 답변
```

답변:

> ACME 고객사의 온보딩은 SSO 설정 문제로 지연되고 있습니다.

틀리지는 않지만 추가 맥락이 부족하다.

8.2 Agentic RAG의 답변

Agentic RAG 흐름:

```
1. ACME 관련 문서 검색
2. 최근 티켓 검색
3. SSO 관련 회의록 검색
4. 담당 팀 로드맵 검색
5. 정보가 충분한지 평가
6. 원인과 해결 방안 정리
```

답변:

> ACME 고객사의 온보딩 지연은 단순한 SSO 설정 문제가 아니라 세 가지 원인이 겹친 결과입니다. 첫째, ACME가 요구한 SSO 설정이 아직 제품 표준 플로우에 포함되지 않았습니다. 둘째, 보안 검토가 완료되지 않아 고객에게 설정 가이드를 제공하지 못하고 있습니다. 셋째, Platform Team이 Billing API를 우선순위로 두면서 SSO 개선 일정이 밀렸습니다. 단기적으로는 ACME 전용 설정 가이드를 제공하고, 중기적으로는 Security Review SLA와 Platform Team 우선순위 조정이 필요합니다.

Agentic RAG는 여러 번 검색하고 판단했기 때문에 더 깊은 답변을 만들 수 있다.

---

9. Agentic RAG와 Graph RAG의 차이

구분	Graph RAG	Agentic RAG
핵심 아이디어	지식 그래프를 활용한 검색	에이전트가 검색과 도구 사용을 동적으로 제어
중심 요소	엔티티, 관계, 그래프	계획, 도구 선택, 반복 검색, 검증
강점	관계와 전체 구조 파악	복잡한 작업 수행과 다단계 조사
주요 질문	“무엇이 무엇과 연결되어 있는가?”	“무엇을 먼저 찾고, 어떤 도구로 검증할 것인가?”
구축 난이도	그래프 구축이 어려움	오케스트레이션과 안정성 관리가 어려움
예시	고객-기능-리스크 관계 분석	정책, 티켓, DB를 종합해 승인 여부 판단

둘은 경쟁 관계가 아니라 함께 사용할 수 있다.

```
Agentic RAG + Graph RAG
= 에이전트가 필요할 때 지식 그래프를 도구처럼 사용한다.
```

예:

```
질문: “ACME 이슈와 관련된 조직적 병목을 찾아줘.”

Agent:
1. 질문 분석
2. Graph Retriever 호출
3. 관련 subgraph 확인
4. 회의록 retriever 호출
```

```
5. 티켓 시스템 API 호출
6. 원인 분석
7. 최종 답변 작성
```

---

10. Agentic RAG가 유용한 경우

Agentic RAG는 다음 상황에 적합하다.

- 질문이 복잡하고 여러 단계의 조사가 필요하다.
- 여러 데이터 소스를 함께 봐야 한다.
- 검색 결과가 충분한지 판단해야 한다.
- 답변 전에 검증이나 비교가 필요하다.
- 문서 검색뿐 아니라 API, DB, 계산, 코드 실행이 필요하다.
- 사용자의 요청이 단순 Q&A가 아니라 작업 수행에 가깝다.

예시:

```
“이 고객 요청을 승인해도 되는지 정책과 계약을 확인해줘.”
“지난달 장애의 원인을 티켓, 회의록, 배포 로그 기준으로 정리해줘.”
“세 개 논문의 방법론과 실험 결과를 비교해줘.”
“이 PR이 내부 코딩 가이드와 충돌하는지 확인해줘.”
```

---

11. Agentic RAG가 과한 경우

다음 경우에는 일반 RAG가 더 낫다.

- 질문이 단순하다.
- 검색 대상이 하나의 문서나 작은 문서 집합이다.
- 빠른 응답 속도가 중요하다.
- 도구 호출 비용을 줄여야 한다.
- 반복 검색이나 계획 수립이 필요 없다.
- 답변 형식이 고정되어 있다.

예:

```
“휴가 정책 알려줘.”
“이 문서 요약해줘.”
“환불 조건이 뭐야?”
```

이런 질문에 Agentic RAG를 쓰면 비용과 지연만 늘어날 수 있다.

---

12. 장점

12.1 복잡한 질문 처리에 강함

Agentic RAG는 질문을 나누고, 여러 검색을 수행하고, 결과를 비교할 수 있다.

```
질문 → 하위 질문 → 검색 → 검토 → 추가 검색 → 종합
```

12.2 여러 도구를 사용할 수 있음

문서 검색뿐 아니라 다양한 도구를 함께 사용할 수 있다.

```
Vector DB
SQL DB
Graph DB
Web Search
Ticket API
Code Interpreter
Calculator
Internal API
```

12.3 검색 실패를 복구할 수 있음

일반 RAG는 첫 검색이 실패하면 답변 품질이 크게 떨어진다.

Agentic RAG는 검색 결과를 평가하고 다시 시도할 수 있다.

```
검색 결과 부족
→ 쿼리 재작성
→ 다른 도구 선택
→ 추가 검색
```

12.4 작업 수행형 AI로 확장하기 좋음

Agentic RAG는 단순 답변을 넘어 작업 수행에 가까워질 수 있다.

예:

```
문서 검토
리포트 작성
이슈 원인 분석
정책 위반 여부 판단
```

```
데이터 조회 후 요약
코드 분석 후 수정 제안
```

---

13. 단점과 위험

13.1 비용과 지연 증가

Agentic RAG는 여러 번 LLM을 호출하고 여러 도구를 사용할 수 있다.

따라서 일반 RAG보다 비용과 응답 시간이 늘어날 수 있다.

13.2 통제하기 어려움

Agent가 어떤 도구를 몇 번 호출할지 동적으로 결정하기 때문에 예측이 어렵다.

문제 예:

```
불필요한 검색 반복
잘못된 도구 선택
너무 많은 토큰 사용
무한 루프에 가까운 행동
```

13.3 보안과 권한 관리가 중요함

Agent가 여러 도구와 데이터 소스에 접근하면 권한 관리가 중요해진다.

특히 다음을 주의해야 한다.

- 사용자가 볼 수 없는 문서를 검색하지 않도록 하기
- API 호출 권한 제한하기
- 민감정보 노출 방지하기
- 도구별 audit log 남기기기
- 프롬프트 인젝션 방어하기

13.4 평가가 어려움

일반 RAG는 검색 결과와 답변을 비교하면 어느 정도 평가할 수 있다.

Agentic RAG는 경로가 매번 달라질 수 있어서 평가가 더 어렵다.

평가 대상:

```
계획이 적절했는가?
도구 선택이 맞았는가?
```

```
검색 쿼리가 좋았는가?
충분히 검증했는가?
최종 답변이 근거에 기반했는가?
비용과 시간이 적절했는가?
```

---

14. 구현 아키텍처 예시

```
[User Question]
      ↓
[Agent / Orchestrator]
      ↓
[Planner]
      ↓
[Tool Router]
      ├─ Vector Retriever
      ├─ Graph Retriever
      ├─ SQL Query Tool
      ├─ Web Search Tool
      ├─ Ticket API
      ├─ Code Tool
      └─ Calculator
      ↓
[Observation / State]
      ↓
[Evaluator]
      ├── 충분함 → Final Answer
      └── 부족함 → Re-plan / Re-query
      ↓
[Final Answer + Citations]
```

---

15. 간단한 의사코드

```python
def agentic_rag_answer(question):
    state = {
        "question": question,
        "evidence": [],
        "steps": [],
        "status": "running"
    }

    # 1. 질문 분석 및 계획 수립
    plan = planner(question)
```

```
    for step in plan:
        # 2. 필요한 도구 선택
        tool = select_tool(step)

        # 3. 도구 실행
        result = tool.run(step.query)

        # 4. 결과 저장
        state["evidence"].append(result)
        state["steps"].append({
            "step": step,
            "tool": tool.name,
            "result": result
        })

        # 5. 정보가 충분한지 평가
        evaluation = evaluate(question, state["evidence"])

        if evaluation.is_sufficient:
            break

        # 6. 부족하면 계획 수정
        if evaluation.needs_replanning:
            plan.extend(replan(question, state))

    # 7. 최종 답변 생성
    answer = generate_answer(question, state["evidence"])
    return answer
```

---

16. 실무 설계 포인트

16.1 도구를 너무 많이 주지 않기

Agent에게 도구를 너무 많이 주면 오히려 잘못된 도구 선택이 늘어난다.

처음에는 다음 정도로 시작하는 것이 좋다.

```
1. 문서 검색 도구
2. 특정 DB 조회 도구
3. 결과 평가 도구
```

이후 필요에 따라 도구를 늘린다.

16.2 명확한 중단 조건 만들기

Agent가 계속 검색하지 않도록 중단 조건을 둔다.

예:

```
최대 도구 호출 5회
최대 검색 쿼리 3개
최대 실행 시간 30초
충분한 근거 3개 이상이면 종료
```

16.3 답변에 근거를 강제하기

Agentic RAG는 그럴듯한 추론을 많이 만들 수 있으므로 근거 기반 답변이 중요하다.

프롬프트 예:

```
답변의 모든 주요 주장에는 반드시 근거 문서나 도구 결과를 연결하라.
근거가 부족한 내용은 추측이라고 표시하라.
```

16.4 도구 호출 로그를 남기기

운영 환경에서는 다음 로그가 중요하다.

```
사용자 질문
Agent의 계획
호출한 도구
검색 쿼리
검색 결과
최종 답변
비용과 latency
```

이 로그가 있어야 디버깅과 평가가 가능하다.

16.5 권한을 도구 레벨에서 통제하기

Agent에게 모든 권한을 주면 위험하다.

권장 방식:

```
사용자 권한 확인
→ 접근 가능한 문서만 검색
→ 도구별 permission check
```

```
→ 민감정보 필터링
→ audit log 저장
```

---

17. 평가 지표

Agentic RAG는 다음 기준으로 평가할 수 있다.

17.1 답변 품질

- 정확한가?
- 질문에 직접 답하는가?
- 필요한 맥락을 포함하는가?
- 근거가 충분한가?

17.2 검색 품질

- 적절한 도구를 선택했는가?
- 검색 쿼리가 적절했는가?
- 중요한 문서를 놓치지 않았는가?
- 불필요한 검색을 줄였는가?

17.3 추론 품질

- 하위 질문 분해가 적절한가?
- 중간 결론이 근거와 일치하는가?
- 모순되는 정보를 잘 처리했는가?

17.4 운영 지표

- 응답 시간
- 도구 호출 횟수
- LLM 호출 횟수
- 토큰 비용
- 실패율
- 재시도율

---

18. Graph RAG와 함께 쓰는 구조

Agentic RAG와 Graph RAG는 함께 쓰면 강력하다.

```
Agentic RAG = 무엇을 할지 판단하는 방식
Graph RAG = 관계 기반으로 검색하는 도구
```

조합 예:

```
[Agent]
  ├─ Vector Search Tool
  ├─ Graph Search Tool
  ├─ SQL Tool
  ├─ Ticket Tool
  └─ Web Search Tool
```

질문:

> “ACME와 Globex의 온보딩 이슈가 같은 원인인지 분석해줘.”

Agent의 행동:

```
1. ACME 관련 문서 검색
2. Globex 관련 문서 검색
3. Graph Search로 공통 노드와 관계 확인
4. 티켓 시스템에서 최근 상태 확인
5. 두 고객의 공통 원인과 차이점 정리
```

이 구조에서는 Graph RAG가 Agentic RAG의 도구 중 하나가 된다.

---

19. 적용 예시

19.1 사내 지식 검색

질문:

> “이 프로젝트가 왜 지연됐는지 관련 문서를 종합해서 설명해줘.”

Agentic RAG는 회의록, 티켓, 로드맵, 의사결정 문서를 순차적으로 검색하고, 부족한 부분을 다시 찾는다.

19.2 고객지원 분석

질문:

> “이 고객의 불만이 계약 리스크로 이어질 가능성이 있어?”

Agentic RAG는 고객 티켓, 계약 조건, 최근 미팅 노트, 제품 로드맵을 함께 확인한다.

19.3 법률 검토

질문:

> “이 계약 변경 요청을 받아들여도 되는지 검토해줘.”

Agentic RAG는 계약서, 내부 정책, 과거 예외 승인 사례를 검색하고 리스크를 정리한다.

19.4 연구 리서치

질문:

> “이 분야에서 최근 방법론들을 비교하고, 어떤 방향이 유망한지 정리해줘.”

Agentic RAG는 논문 검색, 방법론 분류, 성능 비교, 한계점 분석을 단계적으로 수행한다.

19.5 데이터 분석

질문:

> “지난달 고객 이탈 원인을 문서와 수치 데이터 기준으로 분석해줘.”

Agentic RAG는 문서 검색과 SQL 분석을 함께 사용한다.

---

20. 도입 전 체크리스트

Agentic RAG를 도입하기 전에 다음을 확인한다.

- 질문이 단순 검색보다 복잡한가?
- 여러 데이터 소스나 도구가 필요한가?
- 검색 결과가 충분한지 평가해야 하는가?
- 반복 검색이나 쿼리 재작성 필요성이 있는가?
- 도구 호출 비용과 지연을 감당할 수 있는가?
- 도구 권한과 보안 통제가 가능한가?
- Agent의 행동 로그를 남길 수 있는가?
- 실패했을 때 fallback 경로가 있는가?

---

21. 한 문장 요약

Agentic RAG는 에이전트가 질문을 분석하고, 필요한 도구를 선택하고, 검색과 검증을 반복하면서 답변을 만드는 RAG 방식이다.

쉽게 말하면:

> 일반 RAG는 “검색해서 답하는 시스템”이고,
> Agentic RAG는 “스스로 조사 계획을 세우고 필요한 자료를 찾아 답하는 시스템”이다.